5.1 “移近—归约”分析法
5.2 算符优先分析法
5.3 LR分析概述
5.4 LR (0)分析
5.5 SLR (1)分析
5.6 LR (1)分析
5.7 LALR (1)分析
5.8 LR分析对二义文法的应用
5.9 LR分析的错误处理与恢复
5.10 语法分析器的自动生成与YACC

5.3 LR分析概述

5.3.1 LR分析与LR分析器

5.3.2 LR分析器结构、组成与工作原理

5.3.3 LR分析实例

LR分析技术是编译系统中语法分析器实现最常用、最有效的一种分析方法。

(1) 理论上比较完善；

(2) 适用性强，对*G*限定少；

(3) 便于自动生成。

LR分析：一类对源程序串进行自左向右扫描并进行规范归约的语法分析方法。

L R (*k*)

在LR分析的每一步，仅据分析栈当前已经移进和归约的全部语法符号并最多再向前看*k*个输入字符，就能确定适合于文法规则的句柄是否已在栈顶形成，从而立即确定当前的分析动作。

分析模式：最右推导的逆序(规范归约)

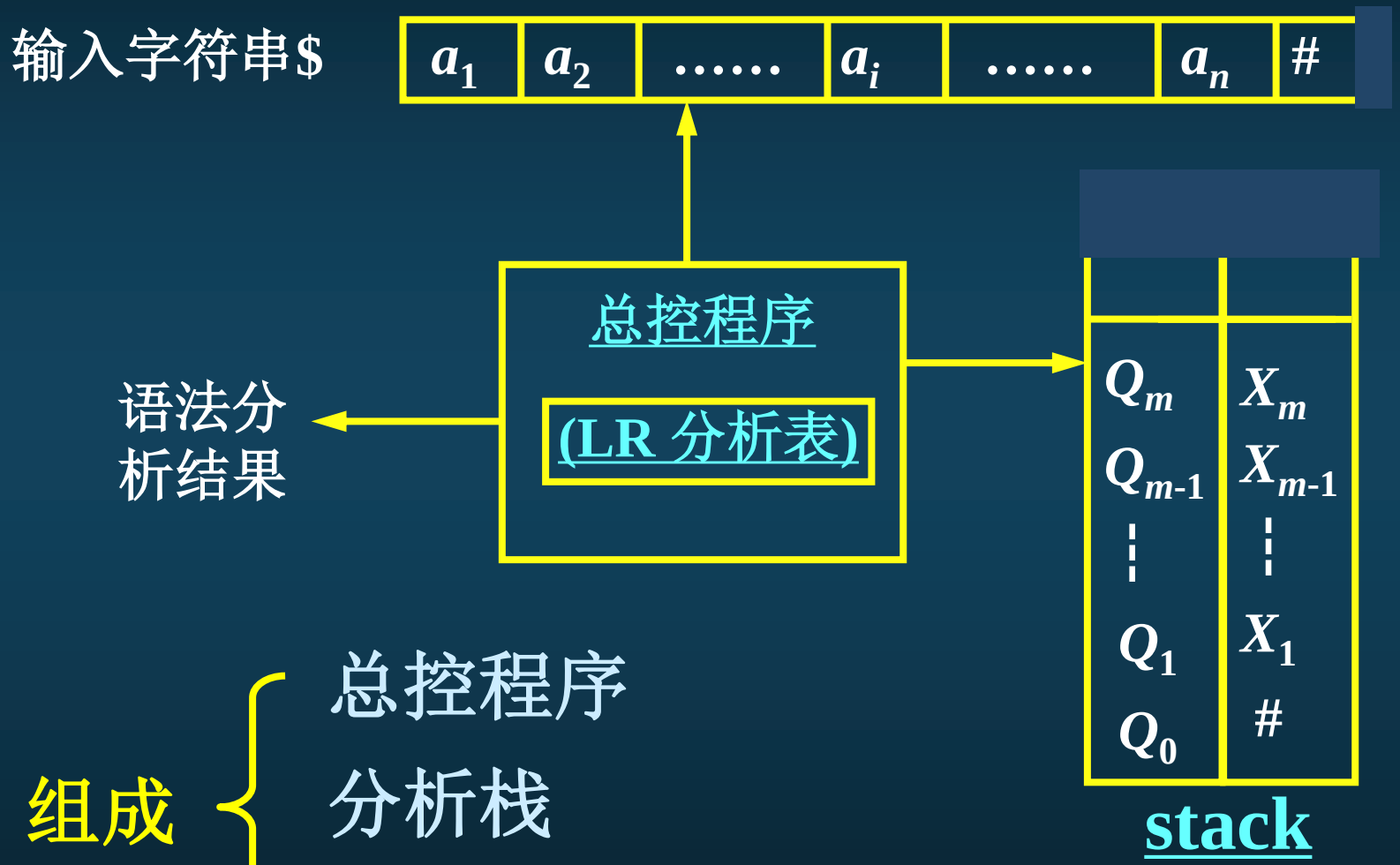
扫描模式：自左向右

5.3.1 LR分析与LR分析器

5.3.2 LR分析器结构、组成与工作原理

5.3.3 LR分析实例

LR分析器逻辑结构



组成
- 总控程序
- 分析栈
- LR分析表

（1）分析栈

辅助完成LR分析的数据结构。

Q_m	X_m
Q_{m-1}	X_{m-1}
⋮	⋮
Q_1	X_1
Q_0	#

stack
- 状态Q_i：记录分析过程中每一步的“历史”和“展望”信息；
- 文法符号X_i：存放分析过程中移进(V_T)和归约 (V_N)的符号；

stack初始化：

Q_0	#

← P

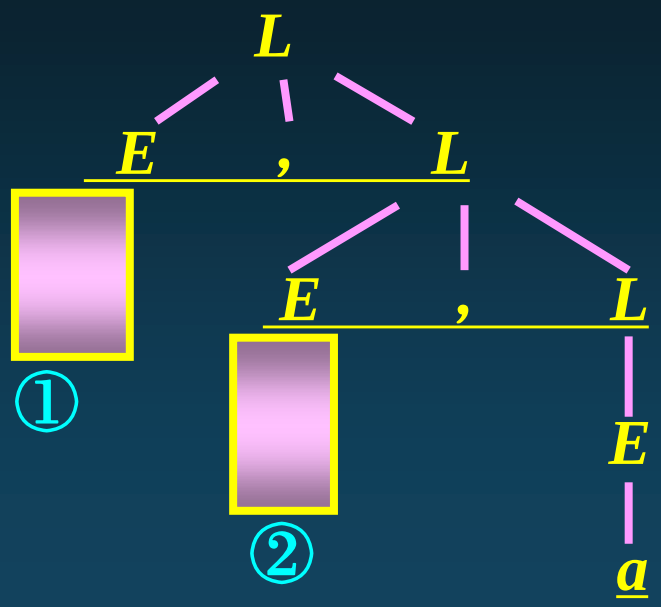
例如，设有文法$G(L)$和$G(L)$的LR分析表

① $L \rightarrow E, L$

② $L \rightarrow E$

③ $E \rightarrow a$

④ $E \rightarrow b$



F

输入字符串 $a, b, a\#$

$b \Rightarrow E$ (归约)

, (移进栈)

$a \Rightarrow E$ (归约)

2	E
5	,
2	E
0	#

P

文法G(L)的LR分析表

状态	ACTION表				GOTO表	
	a	*b*	,	#	*E*	*L*
0	S_3	S_4			2	1
1				acc		
2			S_5	r_2		
3			r_3	r_3		
4			r_4	r_4		
5	S_3	S_4			2	6
6				r_1		

例如，设有文法$G(L)$和

$G(L)$的LR分析表

① $L \rightarrow E, L$

② $L \rightarrow E$

③ $E \rightarrow a$

④ $E \rightarrow b$

F

输入字符串 $a, b, a\#$

$b \Rightarrow E$ (归约)

，(移进栈)

$a \Rightarrow E$ (归约)

2	E
5	,
2	E
0	#

P

（2） LR分析表

LR分析表是LR分析器的核心。

分析表 $\begin{cases} \text{分析动作表(ACTION表)} \\ \text{状态转换表(GOTO表)} \end{cases}$

分析动作表 $\longrightarrow$ $Q\times(V_T\cup\{\#\})$

状态转换表 $\longrightarrow$ $Q\times V_N$

LR 分析表

table / V / state	Action (V_T)					Goto (V_N)			
	V_{T1}	V_{T2}	…	V_{Tn}	#	V_{N1}	V_{N2}	…	V_{Nm}
Q_0	…					…			
Q_1	…					…			
…	…					…			
Q_n	…					…			

状态转换表 (Goto)

state \ V_N	X_1	X_2	…	X_n
Q_0	goto(Q_0,X_1)	goto(Q_0, X_2)	…	goto(Q_0, X_n)
Q_1	goto(Q_1,X_1)	goto(Q_1, X_2)	…	goto(Q_1, X_n)
…	…	…	…	…
Q_m	goto(Q_m,X_1)	goto(Q_m, X_2)	…	goto(Q_m, X_n)

$Q_i \in$状态； $X_i \in$非终结符集；

$$\text{goto}(Q_i, X_i) = \begin{cases} Q_j & \text{(移进：将}Q_j\text{状态压入状态栈)} \\ \text{空} & \text{(不会出现的情况，没有动作)} \end{cases}$$

注意：

goto(Q_i , X_i) 的Q_i、X_i 意指当前栈顶的X_i和次栈顶的Q_i元素。

分析动作表 (Action)

state \ V_T	a_1	a_2	…	a_n
Q_0	action(Q_0,a_1)	action(Q_0,a_2)	…	action(Q_0,a_n)
Q_1	action(Q_1,a_1)	action(Q_1,a_2)	…	action(Q_1,a_n)
…	…	…	…	…
Q_n	action(Q_n,a_1)	action(Q_n,a_2)	…	action(Q_n,a_n)

$Q_i \in$状态； $a_i \in$终结符集或#；

$$\text{action}(Q_i, a_i) = \begin{cases} S_{Q_j} \text{ (移进：将} a_i \text{和} Q_j \text{状态压入栈)} \\ \\ \end{cases}$$

Q_0	Q_1	…	Q_m	
#	X_1	…	X_m	

P

$\$: a_1\ a_2\ \ldots\ a_i a_{i+1} \ldots a_n\ \#$

F

$action(Q_m, a_i) = S_{Qk}$

Q_0	Q_1	…	Q_m	Q_k
#	X_1	…	X_m	a_i

P

$\$: a_1\ a_2\ \ldots\ a_i a_{i+1} \ldots a_n\ \#$

F

$$action(Q_i, a_i) = \begin{cases} S_{Qj} \text{ (移进：将} a_i \text{和} Q_j \text{状态压入栈)} \\ r_j \text{ (归约：用第} j \text{个产生式归约)} \end{cases}$$

$: $a_1\ a_2\ \ldots\ a_i a_{i+1} \ldots a_n$ #

F

Q_0	Q_1	…	Q_{m-r}	Q_{m-r+1}	…	Q_m	
#	X_1	…	X_{m-r}	X_{m-r+1}	…	X_m	

句柄

P

action(Q_m , a_i) = r_j

① 归约。设G第j个产生式为： $A \rightarrow X_{m-r+1} X_{m-r+2} \cdots X_m$

Q_0	Q_1	…	Q_{m-r}		
#	X_1	…	X_{m-r}	A	

P

② 查goto表。 goto(Q_{m-r} , A) = Q_j

Q_0	Q_1	…	Q_{m-r}	Q_j	
#	X_1	…	X_{m-r}	A	

P

$: $a_1\ a_2\ \ldots\ a_i a_{i+1} \ldots a_n$ #　（扫描指针不变）

F

$$\text{action}(Q_i, a_i) = \begin{cases} S_{Qj} \text{（移进：将} a_i \text{和第} j \text{个状态压入栈} \\ r_j \text{（归约：用第} j \text{个产生式归约）} \\ \text{acc（接受：分析成功）} \\ \text{error（出错：语法错，调出错处理程序）} \end{cases}$$

文法G(L)的LR分析表

状态	ACTION表				GOTO表	
	a	*b*	,	#	*E*	*L*
0	S_3	S_4			2	1
1				acc		
2			S_5	r_2		
3			r_3	r_3		
4			r_4	r_4		
5	S_3	S_4			2	6
6				r_1		

分析

（3） LR分析总控程序

① 分析开始，将初始状态Q_0及输入字符串左界符“＃”压入分析栈； // 初始化

②据当前分析栈栈顶Q_m，当前输入符号a_i查action 表：

i)若action(Q_m , a_i)＝S_{Qj}，完成移进动作；

ii)若action (Q_m , a_i)＝r_j，完成归约动作。

iii)若action(Q_m , a_i)＝acc，分析成功；

iv)若action(Q_m , a_i)＝error，出错处理。

③转②。

5.3.1 LR分析与LR分析器

5.3.2 LR分析器结构、组成与工作原理

5.3.3 LR分析实例

例5.10 设有文法$G(L)$和$G(L)$的LR分析表。

① $L \rightarrow E, L$

② $L \rightarrow E$

③ $E \rightarrow a$

④ $E \rightarrow b$

输入字符串 $a, b, a\#$

步骤	栈中状态	输入符号串	栈中符号	分析动作
1	0	a,b,a #	#	S_3
2	03	,b,a #	# a	r_3(用规则③$E \to a$归约)
3	02	,b,a #	# E	S_5
4	025	b,a #	# E,	S_4
5	0254	,a #	# E,b	r_4(用规则④$E \to b$归约)
6	0252	,a #	# E,E	S_5
7	02525	a #	# E,E,	S_3
8	025253	#	# E,E,a	r_3(用规则③$E \to a$归约)
9	025252	#	# E,E,E	r_2 (用规则②$L \to E$归约)
10	025256	#	# E,E,L	r_1 (用规则①$L \to E,L$归约)
11	0256	#	# E,L	r_1 (用规则①$L \to E,L$归约)
12	01	#	# L	acc

查表

对比

☺ 综述：

(1) 处理直观简单；

(2) 基本实现思想：引入状态，状态埋伏了分析的“历史”和“展望”信息；

(3) 应用范围广，对*G*限定少；

(4) LR分析的关键 — LR分析表：集成了全部分析信息。

5.1 “移近—归约”分析法

5.2 算符优先分析法

5.3 LR分析概述

5.4 LR (0)分析

5.5 SLR (1)分析

5.6 LR (1)分析

5.7 LALR (1)分析

5.8 LR分析对二义文法的应用

5.9 LR分析的错误处理与恢复

5.10 语法分析器的自动生成与YACC

5.4.1 LR(0)分析实现思想

5.4.2 构造LR(0)项目集规范族

5.4.3 LR (0)分析表的构造

- **定义5.6 （活前缀）**

规范句型的不含句柄之后任何符号的前缀，称为该句型的活前缀。

注意：

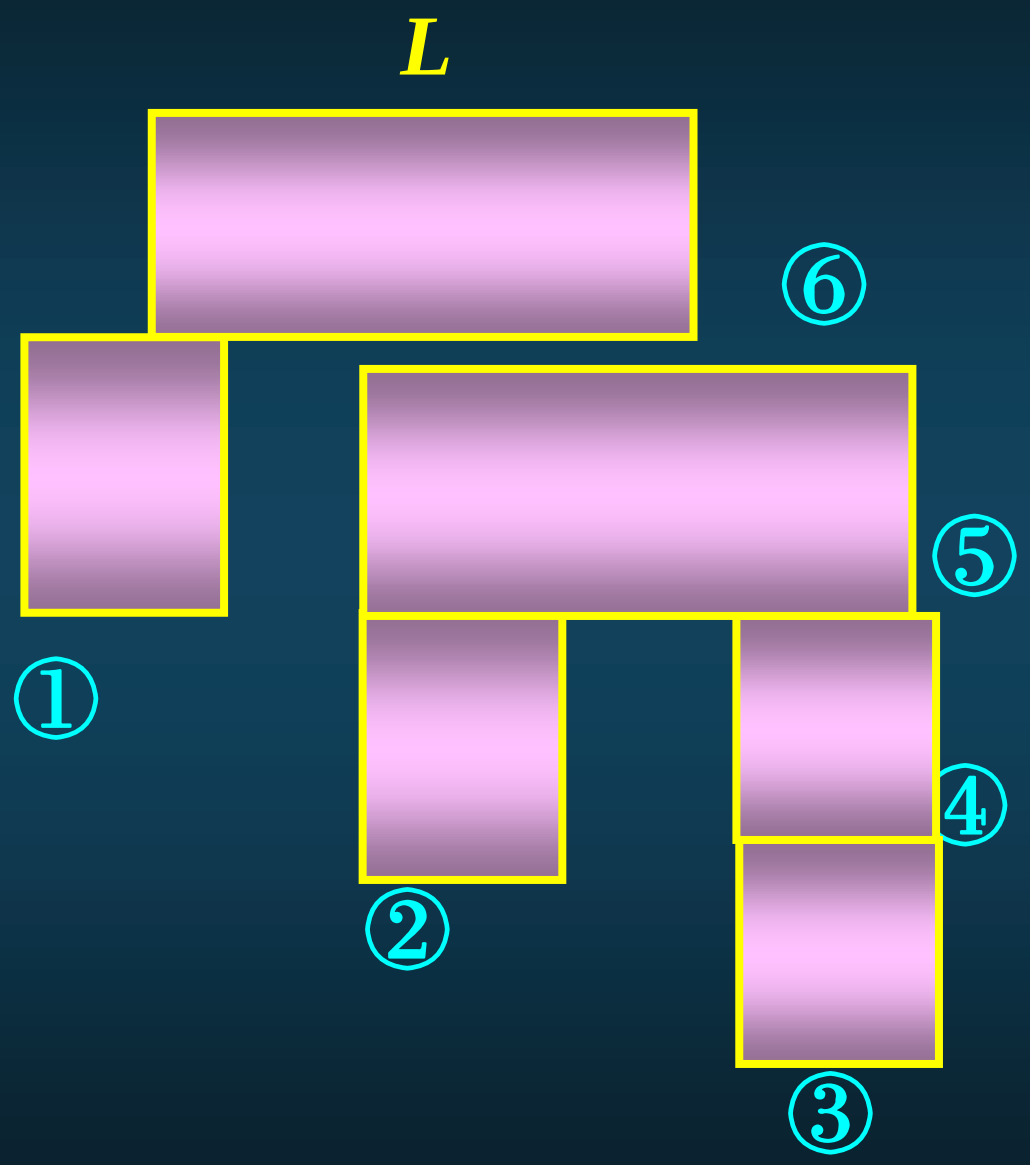
(1) $\exists\ S \overset{*}{\underset{R}{\Rightarrow}} \alpha A\omega \underset{R}{\Rightarrow} \alpha\beta\omega$，若串$\gamma$是$\alpha\beta$的前缀，则$\gamma$是活前缀；当$\gamma=\alpha\beta$，则$\gamma$是可归前缀；

(2) 活前缀特点: 不含句柄之后的任何符号；

(3) LR分析中栈中符号始终是活前缀，当构成刚好包含句柄的活前缀(可归前缀)时，实施归约。

步骤	栈中状态	输入符号串	栈中符号	分析动作
1	0	*a*,*b*,*a* #	#	S_3
2	03	,*b*,*a* #	# *a*	r_3(用规则③*E*→*a*归约)
3	02	,*b*,*a* #	# *E*	S_5
4	025	*b*,*a* #	# *E*,	S_4
5	0254	,*a* #	# *E*,*b*	r_4 (用规则④*E*→*b*归约)
6	0252	,*a* #	# *E*,*E*	S_5
7	02525	*a* #	# *E*,*E*,	S_3
8	025253	#	# *E*,*E*,*a*	r_3 (用规则③*E*→*a*归约)
9	025252	#	# *E*,*E*,*E*	r2 (用规则②*L*→*E*归约)
10	025256	#	# *E*,*E*,*L*	r1 (用规则①*L*→*E*,*L*归约)
11	0256	#	# *E*,*L*	r1 (用规则①归约)
12	01	#	# *L*	acc

对比



对比

- **定义5.7 （LR(0)项目）**

在文法G的每个产生式的右部(候选式)的任何位置上添加一个圆点，这样构成的每个产生式称为LR(0)项目。

约定：若产生式形式为$A \rightarrow \varepsilon$则其LR(0)项目为：$\boxed{A \rightarrow \cdot}$

圆点的位置标记相应的候选式已被匹配（放入栈中）了多少。

例5.11 设文法$G(S)$

$$S \to A \mid B \qquad A \to aA \mid b \mid \varepsilon \qquad B \to c$$

则$G(S)$的LR(0)项目有：

$S \to \cdot A$ $\quad S \to A \cdot$

$S \to \cdot B$ $\quad S \to B \cdot$

$A \to \cdot aA$ $\quad A \to a \cdot A$ $\quad A \to aA \cdot$

$A \to \cdot b$ $\quad A \to b \cdot$

$A \to \cdot$

$B \to \cdot c$ $\quad B \to c \cdot$

LR (0) 项目分类

（1） 归约项目：$A \rightarrow \alpha \cdot$

句柄α恰好包含在栈中，即当前栈中的内容构成了所期望的刚好含句柄的活前缀，应按 $A \rightarrow \alpha$进行归约。

（2） 接受项目： $S' \rightarrow \alpha \cdot$ （S' 是开始符号）

其中S' 是文法惟一的开始符号。这类LR(0)项目实际是特殊的归约项目，表示分析栈中内容恰好为α，用$S' \rightarrow \alpha$ 进行归约，则整个分析成功。

（3） 移进项目： $A\rightarrow \alpha \cdot a\beta\ (a \in V_T)$

这类LR(0)项目表示分析栈中是不完全包含句柄的活前缀，为构成恰好含有句柄$\alpha a\beta$的活前缀，还需将a 移进分析栈。

（4） 待约项目： $A\rightarrow \alpha \cdot B\beta \qquad (B\in V_N)$

这类LR(0)项目表示分析栈中是不完全包含句柄的活前缀，为构成恰好有句柄$\alpha B\beta$的活前缀，应先把当前输入字符串中的相应内容先归约到B。

例5.11 设文法$G(S)$

$$S'\to S \quad S\to A \mid B \quad A\to aA \mid b \mid \varepsilon \quad B\to c$$

则$G(S)$的LR(0)项目有：

$S'\to \cdot S$

$S\to \cdot A$

$S\to \cdot B$

$S'\to S \cdot$

$S\to A \cdot$

$S\to B \cdot$

$A\to \cdot aA$

$A\to \cdot b$

$A\to a \cdot A$

$A\to aA \cdot$

$A\to b \cdot$

$A\to \cdot$

$B\to \cdot c$

$B\to c \cdot$

构造识别文法*G*的所有活前缀的非确定有限自动机：

① 规定文法开始符号在左侧的产生式(设$S' \rightarrow A$)的第一个LR(0)项目(即$S' \rightarrow \cdot A$)为NFA的唯一初态；

② 令所有LR(0)项目分别对应NFA的一个状态且LR(0)项目为归约项目的对应状态为终态。

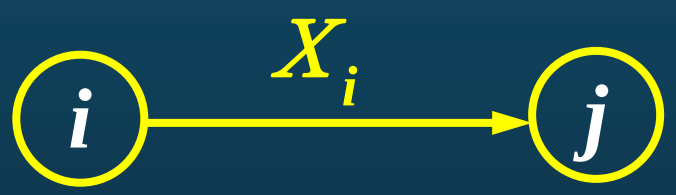
圆点在候选式的最左侧

③若状态 i 和状态j出自同一文法G的产生式且两个状态LR(0) 项目的圆点只相差一个位置，即：

i 为: $X \rightarrow X_1 X_2 \ldots X_{i-1} \cdot X_i \ldots X_n$

j为: $X \rightarrow X_1 X_2 \ldots X_i \cdot X_{i+1} \ldots X_n$

则从状态 i 引一条标记为 X_i的弧到状态j.



若状态 i 为待约项目(设为 $X \rightarrow \alpha \cdot A\beta$)，

则从状态 i 引ε弧到所有$A \rightarrow \cdot \gamma$的状态。

返回

识别文法G的所有活前缀的NFA

NFA $M=(Q, \Sigma, f, q_0, Z)$

LR(0)项目

G的V

开始符号的第1个LR(0)项目

归约项目

若 i 为: $X \to X_1 X_2 \ldots X_{i-1} \cdot X_i \ldots X_n$

j为: $X \to X_1 X_2 \ldots X_i \cdot X_{i+1} \ldots X_n$

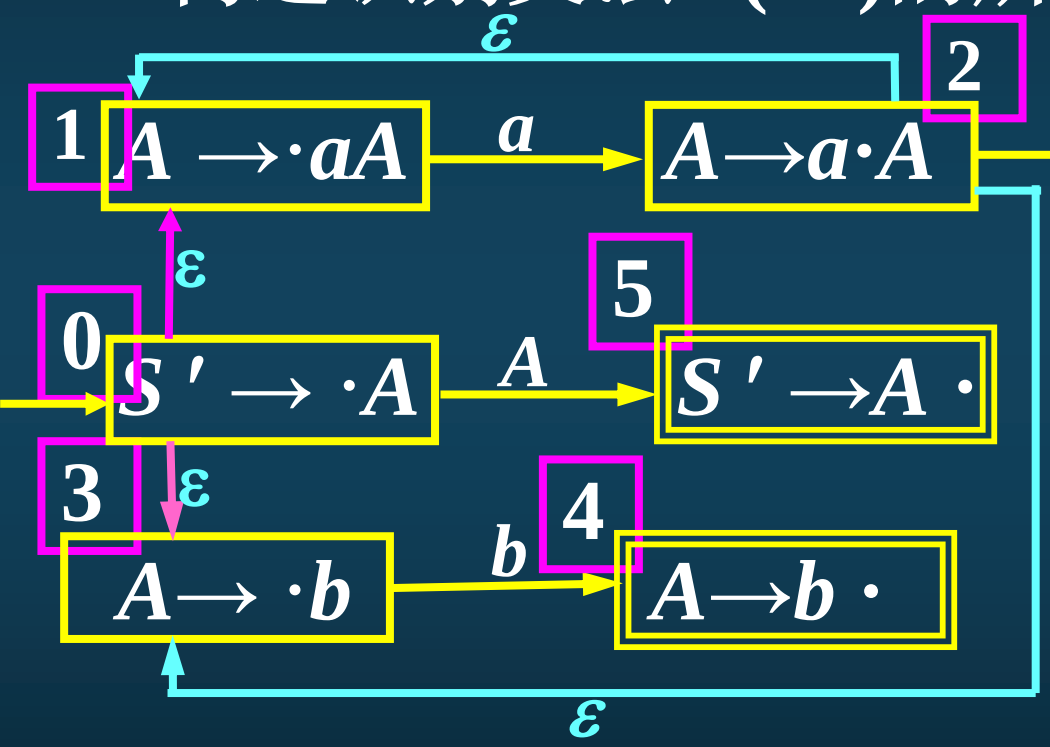
则$j \in f(i, X_i)$。

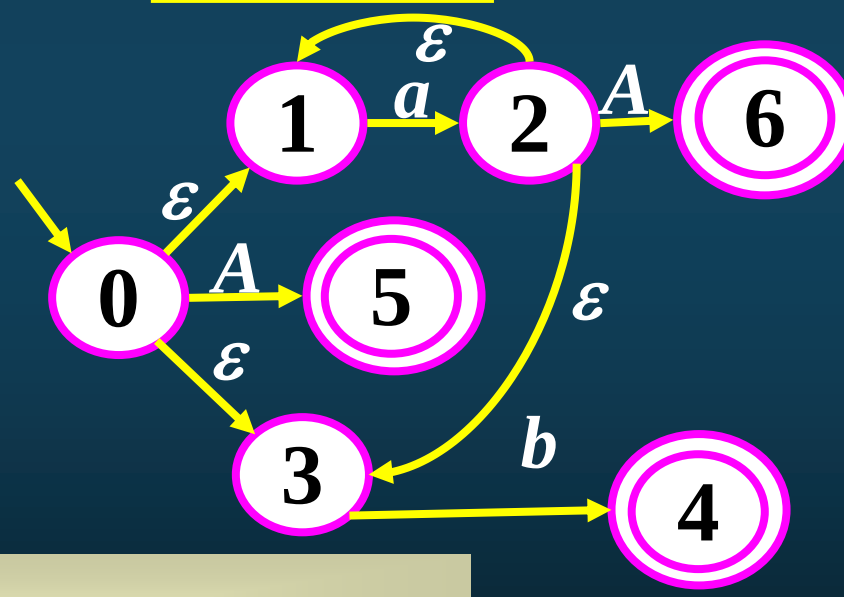
若状态i为待约项目(设 $X \to \alpha \cdot A\beta$)，则所有$A \to \cdot \gamma$的状态包含于$f(i, \varepsilon)$。

例5.12 设文法$G(S')$

$S' \to A \qquad A \to aA \mid b$

构造识别文法$G(S')$的所有活前缀的NFA。

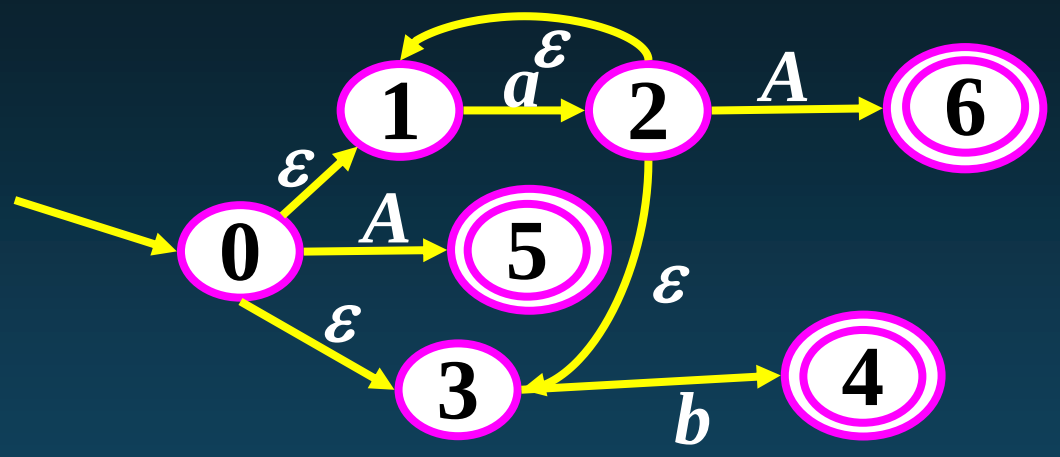




注意：

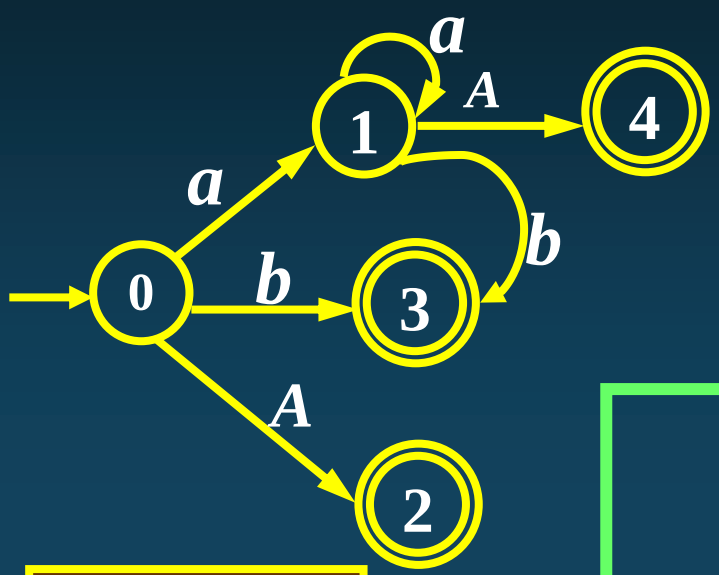
4,5,6为句柄识别态；5为句子识别态。

识别文法$G(S')$的所有活前缀的DFA：



	I	I_a	I_A	I_b
0	{0,1,3}	{2,1,3}	{5}	{4}
1	{2,1,3}	{2,1,3}	{6}	{4}
2*	{5}	∅	∅	∅
3*	{4}	∅	∅	∅
4*	{6}	∅	∅	∅

识别活前缀的DFA

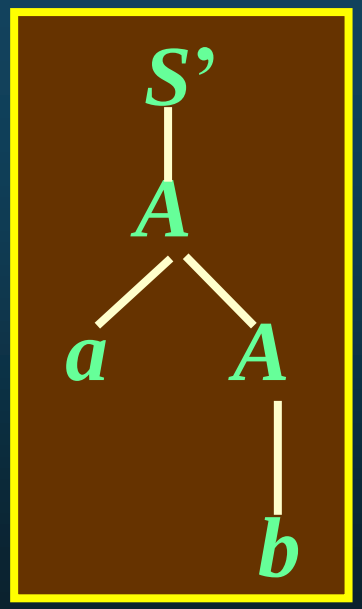


句子*ab*的分析：

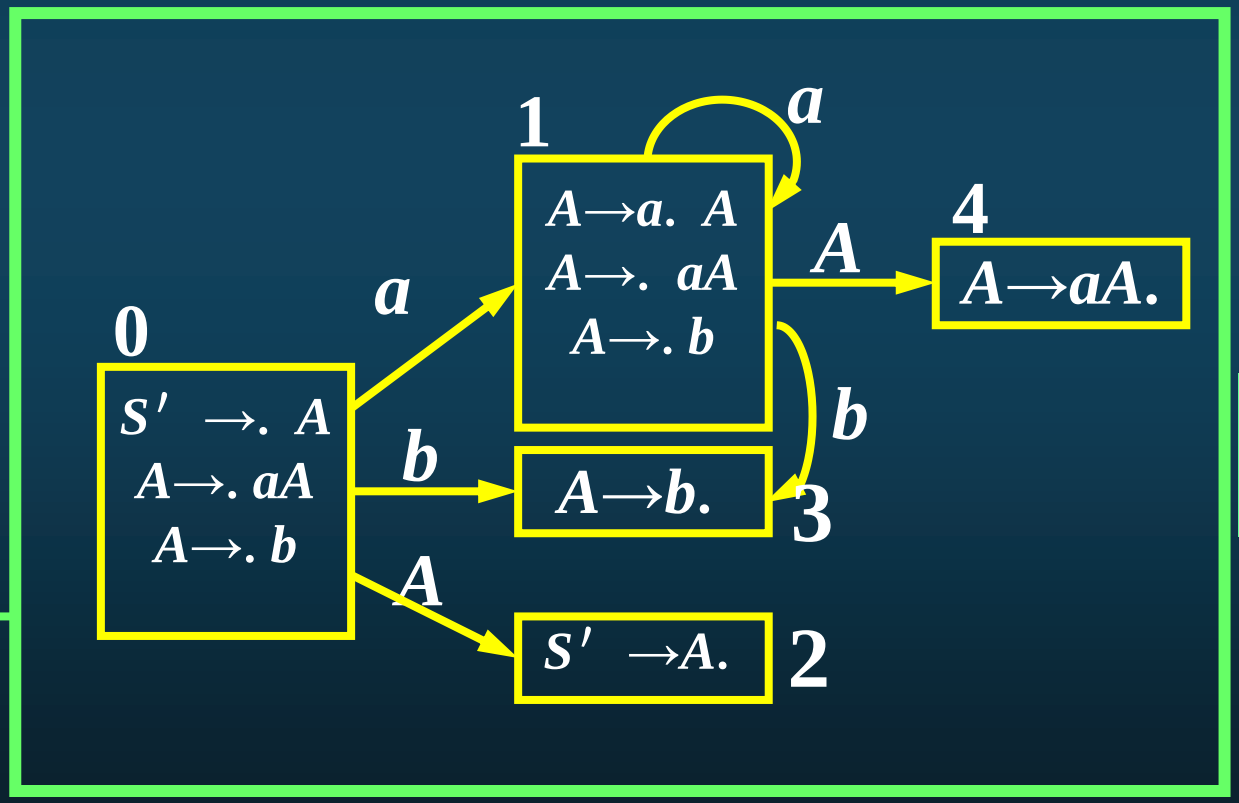
0→1 →3 *ab* (*b*归约到*A*)

0→1 →4 *aA* (*aA*归约到*A*)

0→2 *A* (*A*归约到*S'*)



C



返回

5.4.1　LR(0)分析实现思想

5.4.2　构造LR(0)项目集规范族

5.4.3　LR (0)分析表的构造

- **定义5.9**

识别文法*G*的活前缀的DFA，其状态中包含的LR(0)项目集的全体称为文法*G*的LR(0)项目集规范族。

例5.12文法*G*(*S*′)的LR(0)项目集规范族*C*为：

$C=(\{S'\to\cdot A,\ A\to\cdot aA,\ A\to\cdot b\}$

$\{A\to a\cdot A,\ A\to\cdot aA,\ A\to\cdot b\}$

$\{A\to b\cdot\}\ \{S'\to A\cdot\}\ \{A\to aA\cdot\})$

查看

构造LR(0)项目集规范族的方法 (之一)

第一步： 构造文法*G*的LR(0)项目；

第二步： 基于LR(0)项目，构造识别文法*G*所有活前缀的NFA；

第三步： NFA确定化为DFA，该DFA的各个状态所包含的项目集的集合，即构成了*G*的LR(0)项目集规范族。

构造LR(0)项目集规范族的方法（之二）

引入两个函数
- closure（项目集闭包）函数
- GO（项目集转移）函数

项目集闭包

假定 I 是文法G 的任一项目集，则构造I的闭包closure(I)的方法如下：

① I中的每一个项目皆属于closure(I)；

② 若形如$A\rightarrow\alpha\cdot B\beta$ ($B\in V_N$) 的项目属于I，则对G中的任何产生式 $B\rightarrow\gamma$的第一个LR(0)项目（$B\rightarrow\cdot\gamma$）也属于closure(I)；

③ 重复上述步骤，直至不再有新的项目加入closure(I)为止；

项目集转换函数GO(I , X) :

- **定义5.10**

若 I 是文法G的一个项目集，X为G 的符号，则GO(I , X)＝closure(J)。

其中

J={形如$A \to \alpha X \cdot \beta$的项目|$\exists A \to \alpha \cdot X\beta \in I$}。

构造LR(0)项目集规范族的方法（之二）

亦作为方法一的第一步

第一步：拓广文法。

为使接受状态易于识别，对于不仅在一个产生式左端出现文法的开始符号的情况,要对文法进行拓广。设G是一文法，S是它的开始符号，则将产生式$S' \rightarrow S$加入到G中构成新的文法G'，S'为 G'的开始符号，G'称为G的拓广文法。（使文法开始符号的侯选式惟一）

例如，设文法$G(S)$为：

$$\boxed{S \to A \mid B} \qquad A \to aA \mid b \mid \varepsilon \quad B \to c$$

则文法$G(S)$的拓广文法$G'(S')$ 为

$$S' \to S$$

$$S \to A \mid B$$

$$A \to aA \mid b \mid \varepsilon$$

$$B \to c$$

第二步： 构造文法G的LR(0)项目集规范族。

利用函数closure和GO，完成LR(0)项目集规范族的构造：

基本项目

itemsets(G')

{ C={closure{ $S' \rightarrow \cdot S$ }};

do{

if (对C的每个项目集 I 和每个文法符号X,

若GO(I, X) 非空且不在C中)

把 GO(I, X)加入C中;

} while (没有更多的项目集可以加入C);

}

利用构造LR(0)项目集规范族的方法 (之二)求如下文法$G(S')$的项目集规范族

$S' \to A \qquad A \to aA \mid b$

开始符号S'的候选式唯一，不用拓广文法；

closure($S' \to .A$)={$S' \to .A$，$A \to .aA \mid .b$}$=I_0$

GO(I_0,A)=closure($S' \to A.$)={$S' \to A.$}$=I_1$

GO(I_0,a)=closure($A \to a.A$)={$A \to a.A$，$A \to .aA \mid .b$}$=I_2$

GO(I_0,b)=closure($A \to b.$)={$A \to b.$}$=I_3$

GO(I_2,A)=closure($A \to aA.$)={$A \to aA.$}$=I_4$

GO(I_2,a)=closure($A \to a.A$)$=I_2$

GO(I_2,b)=closure($A \to b.$)$=I_3$

求得项目集规范族为：

$\{I_0, I_1, I_2, I_3, I_4\}$

$=\{\{S' \to .A, A \to .aA \mid .b\}, \{S' \to A.\},$

$\{A \to a.A, A \to .aA \mid .b\}, \{A \to b.\}, \{A \to aA.\}\}$

项目集GO函数为：

	a	b	A
0	2	3	1
1			
2	2	3	4
3			
4			

应用求项目集规范族的方法二直接得出识别活前缀的DFA＝{Q，Σ，f，q_0，Z}。

1. $Q=C$

2. $\Sigma=V$

3. f＝GO函数

4. q_0为基本项目的闭包对应的状态。

5. 包含归约项目的项目集构成终态集Z 。

应用求项目集规范族的方法二直接求出识别活前缀的DFA＝{Q，V，f，0，Z}。

1. 拓广文法。使文法开始符号的侯选式惟一。
2. 利用函数closure和GO，构造DFA：

初始状态0＝{ closure ($S' \rightarrow \cdot S$) }

do{

if (对每个状态i 和每个文法符号X

若GO(i, X)=j非空且不在Q中)

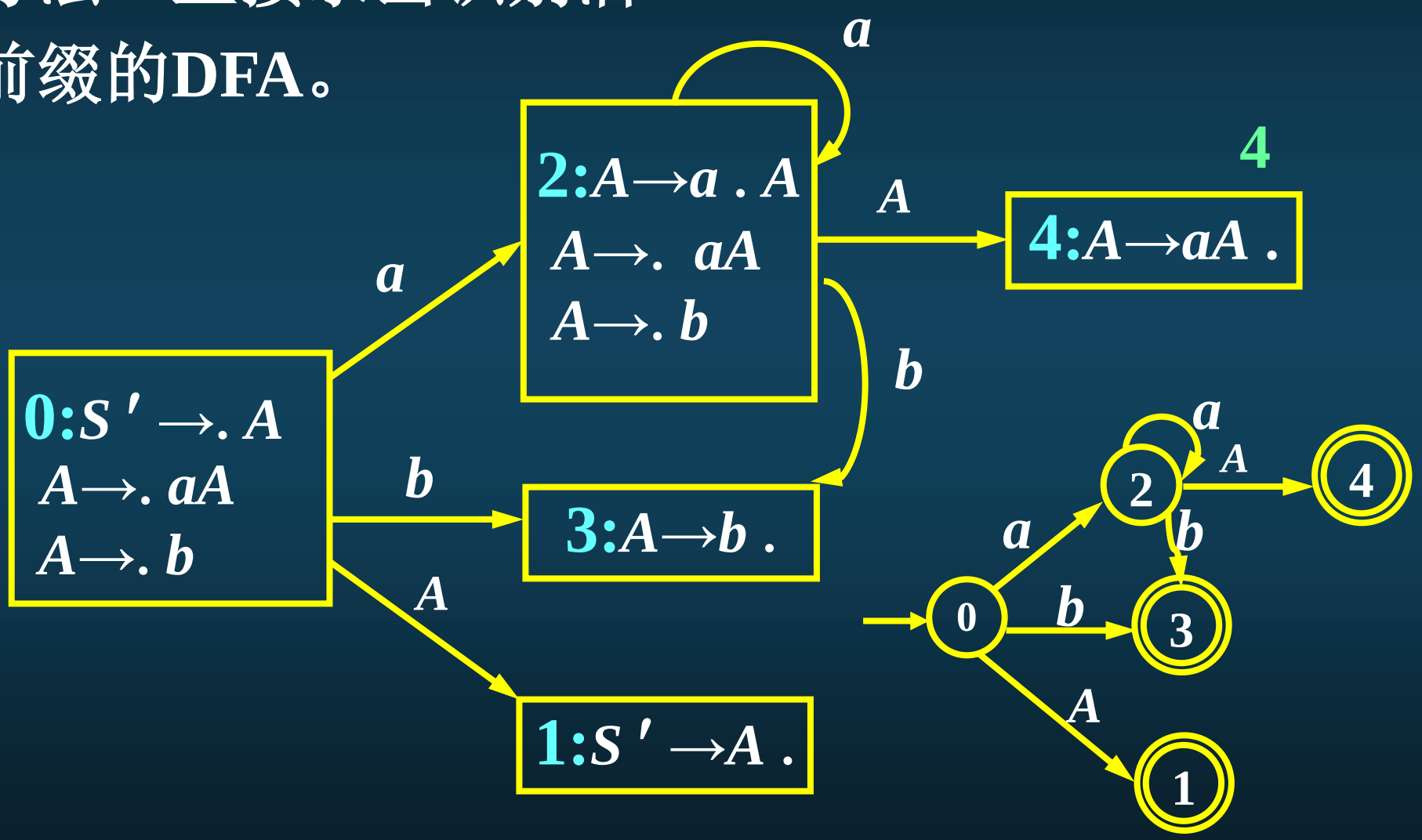
把j加入Q中且$f(i,X)=j$;

} while (没有更多的状态可以加入Q); }

3. 包含归约项目的状态为终态。

$S' \to A \qquad A \to aA \mid b$

应用求项目集规范族的方法二直接求出识别活前缀的DFA。



5.4.1　LR(0)分析实现思想

5.4.2　构造LR(0)项目集规范族

5.4.3　LR (0)分析表的构造

- **定义5.11**

若一个文法*G*的识别活前缀的DFA的每一个状态(项目集规范族中的的每一个LR(0)项目集)中不存在

① 即含移进项目又含归约项目；

和 ② 含有多个归约项目；

则每个项目集的项目相容，称*G*是一个LR(0)文法。

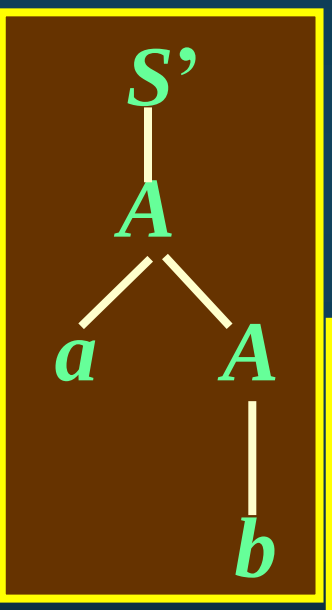
句子*ab*的归约：

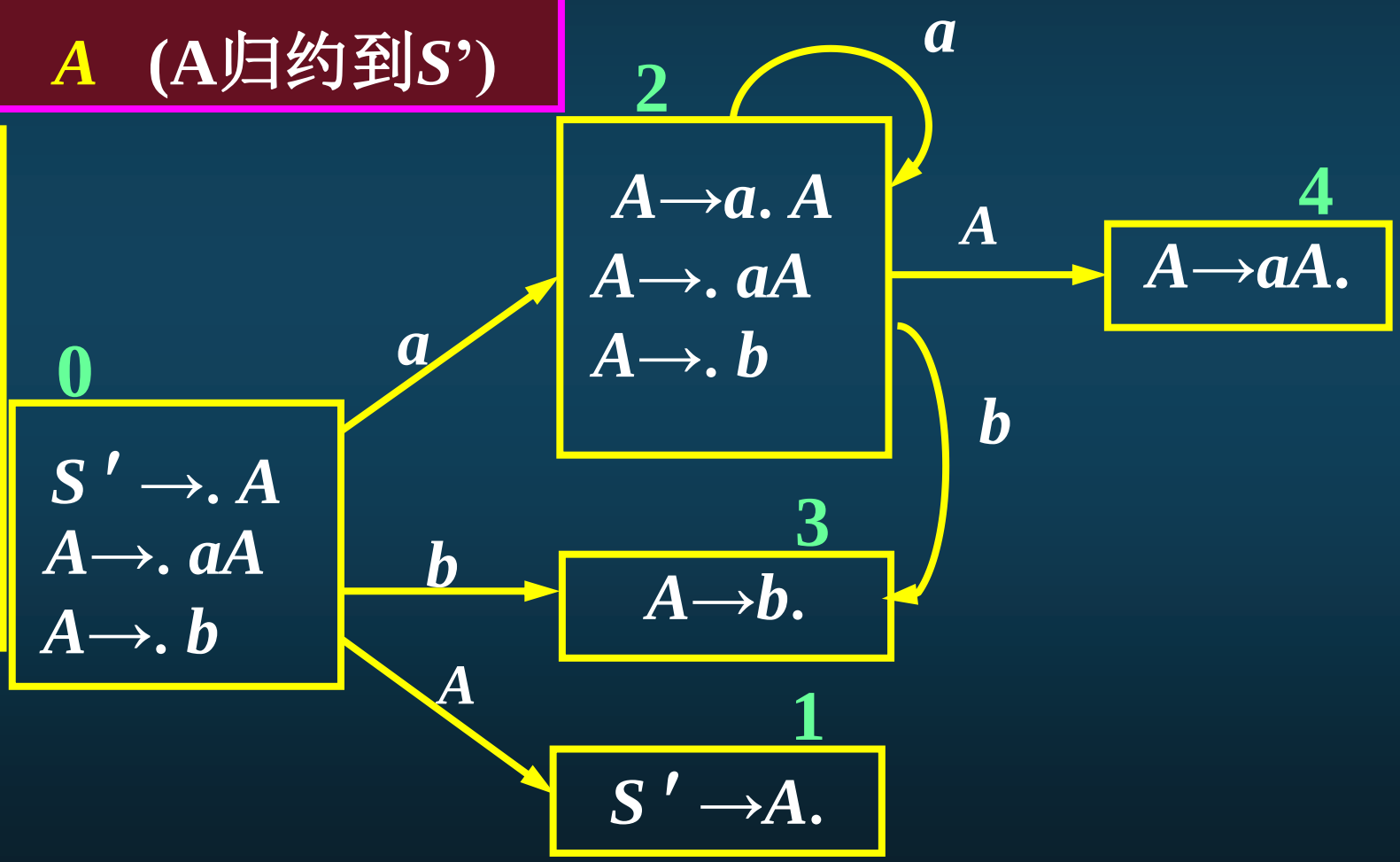
0→2 →3 *ab* (*b*归约到*A*)

0→2 →4 *aA* (*aA*归约到*A*)

0→1 *A* (A归约到*S'*)

$S' \to A \qquad A \to aA \mid b$





识别活前缀的DFA与LR(0)分析的对比

① 对应分析表中action(M, a)＝S_N ($a \in V_T$)，在DFA中为从状态M出发，经过一条a弧到达状态 N；

② 对应分析表中action(M, a)＝r_n ($a \in V_T$)，在DFA中，应对应于归约状态，该状态中的文法产生式编号为 n ；

③ 对分析表中GOTO(M，B)＝N ($B \in V_N$)，在DFA中为从状态M出发，经过一条B弧到达状态 N；

④ 对分析表中action(S_k , #)＝ “acc”,其中k是DFA中的文法的开始符号对应的归约项目的状态。

从识别活前缀的DFA构造LR(0)分析表

- 算法5.6‘ 构造LR(0)分析表

输入：文法G识别活前缀的DFA。

输出：文法G的LR(0)分析表

方法：

分析表的状态集为DFA的状态集

设为$Q=\{0, 1, 2, \ldots n\}$

① 若$f(K, a)=j\ (a\in V_T)$，则置

action$(K, a)=S_j$；

- 算法5.6’ 构造LR(0)分析表 （续）

②若$f(k, A)=j$ ($A\in V_N$)，则置

GOTO(K，A)＝j；

③若$A\rightarrow \alpha\cdot \in k$，则对所有终结符$a$和结束符“＃”，

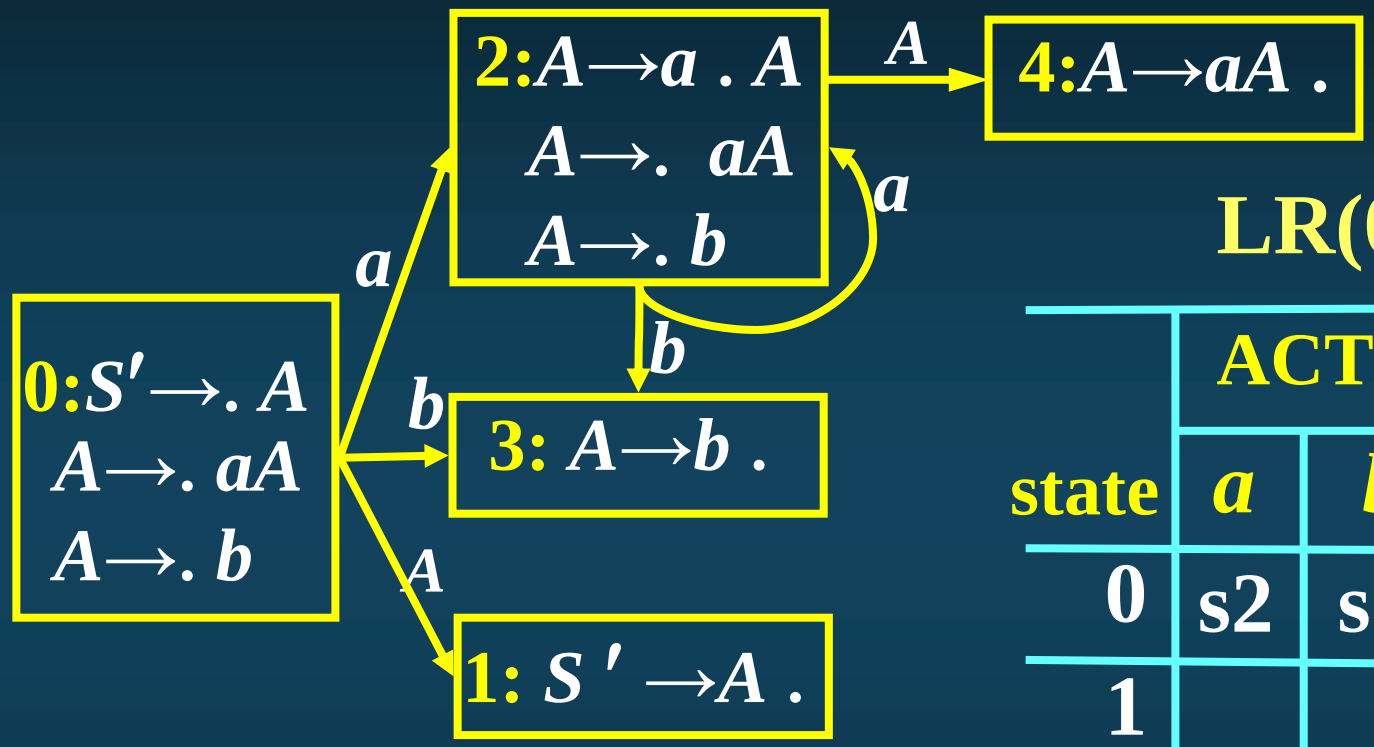
置action(K, a)＝r_j和 action(K, ＃)＝r_j。

(其中假设产生式$A\rightarrow\alpha\cdot$是文法第 j 个产生式)

④ 若$S'\rightarrow S\cdot \in k$，则置action($K$, ＃)＝acc;

⑤ 表中空白置出错标志。

例:对例5.12的文法G(S ′)有识别活前缀的DFA如下



$S' \to A$

$A \to aA$ ① | b ②

LR(0)分析表

	ACTION			GOTO
state	*a*	*b*	#	*A*
0	s2	s3		1
1			acc	
2	s2	s3		4
3	r2	r2	r2	
4	r1	r1	r1	

从LR(0)项目集规范族C和GO函数构造LR(0)分析表

- 算法5.6 构造LR(0)分析表

输入：文法G和文法G的LR(0)项目集规范族C和GO函数

输出：文法G的LR(0)分析表

方法：

设 $C=\{I_0, I_1, \ldots, I_n\}$，每个项目集 I_k 的下标 K作为分析器的状态。

① 若GO $(I_k, a)=I_j (a\in V_T)$，则置 action$(K, a)=S_j$；

- **算法5.6 构造LR(0)分析表 （续）**

② 若GO (I_k ,A)＝I_j ($A\in V_N$)，则置

GOTO(K，A)＝j ；

③ 若$A\rightarrow \alpha\cdot \in I_k$，则对所有终结符$a$和结束符“＃”，置action($K$, a)＝r_j和 action(K , ＃)＝r_j。

(其中假设产生式$A\rightarrow\alpha\cdot$是文法第 j 个产生式)

④ 若$S'\rightarrow S\cdot \in I_k$，则置action($K$, ＃)＝acc;

⑤ 表中空白置出错标志。

对例5.12的文法$G(S')$有LR(0)项目集规范族及GO函数为：

I_0： $\{S' \to \cdot A,\ A \to \cdot aA,\ A \to \cdot b\}$

I_1： $\{S' \to A \cdot\}$

I_2： $\{A \to a \cdot A,\ A \to \cdot aA,\ A \to \cdot b\}$

I_3： $\{A \to b \cdot\}$

I_4： $\{A \to aA \cdot\}$

$S' \to A$

$A \to aA$ ① | b ②

GO函数：

	a	b	A
0	2	3	1
1			
2	2	3	4
3			
4			

文法$G(S')$的LR(0)分析表

state	ACTION表			GOTO表
	a	b	#	A
0	s2	s3		1
1			acc	
2	s2	s3		4
3	r2	r2	r2	
4	r1	r1	r1	

5.1 “移近—归约”分析法

5.2 算符优先分析法

5.3 LR分析鸟瞰

5.4 LR (0)分析

5.5 SLR (1)分析

5.6 LR (1)分析

5.7 LALR (1)分析

5.8 LR分析对二义文法的应用

5.9 LR分析的错误处理与恢复

5.10 语法分析器的自动生成与YACC

例5.13　设有文法*G*，试构造*G*的LR(0)分析表。

G：　$A \to aA \mid a$

首先文法*G*拓广为

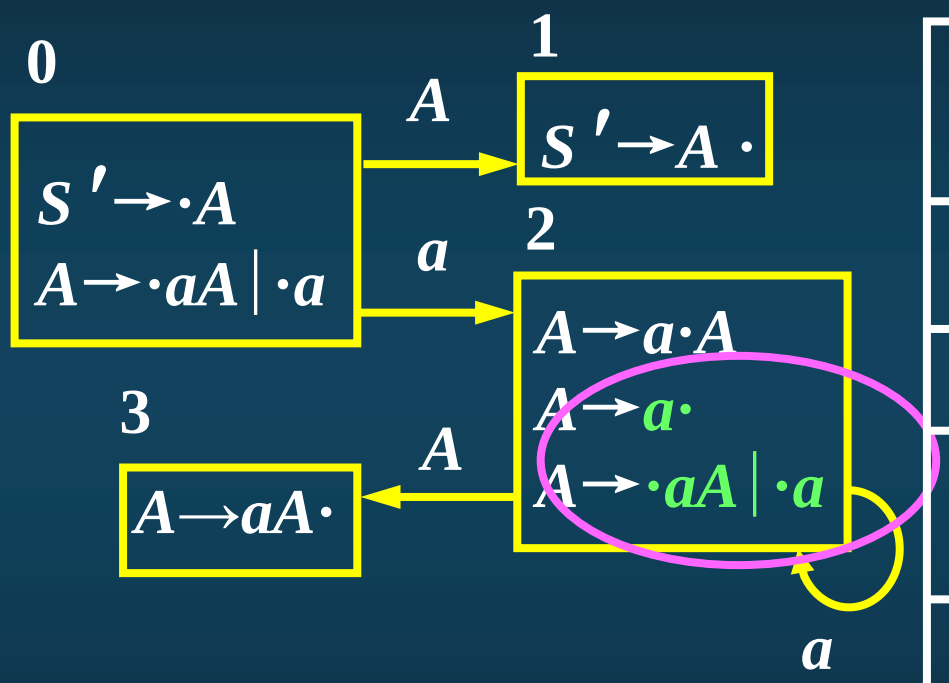
G'：　$S' \to A$

$A \to aA$　①

$A \to a$　②

构造文法G′的识别活前缀的DFA

$S' \to A$
$A \to aA$ ①
$A \to a$ ②



state	action		goto
	a	#	A
0	S_2		1
1		acc	
2	r_2 S_2	r_2	3
3	r_1	r_1	

∴ 文法G′是非LR(0)文法

在识别活前缀的DFA某一状态(项目集规范族的某一项目集）中，若既含有圆点不在最后的移进项目，又含有圆点在最后的归约项目，则称该项目集存在移进－归约冲突。若含有两个或两个以上圆点在最后的归约项目，则称该项目集存在归约－归约冲突。

注意：

冲突情况都与归约动作相关。

- 分析：

LR(0)分析表构造算法对含有归约项目 $A\rightarrow\beta\cdot$ 的项目集（状态）I_i，不管当前输入符号为何，皆把action子表相应于状态 i的那一行的诸元素都指定为 r_j（其中j 为产生式$A\rightarrow\beta\cdot$ 的编号)。

	a_1	a_2	…	a_n
…				
i	r_j	r_j	…	r_j
…				

$$\text{action}(i, a) = \{A \to \beta\} \quad (\forall a \in V_T \cup \{\#\})$$

归约未必有效！

∵ 可能不存在一个规范句型为…Aa…形式。

∴ 用$A \to \beta$归约不一定有效。

- 定义4.5 （回顾）

设上下文无关文法G，S是文法的开始符号，对于文法G的任何非终结符A

$$\text{FOLLOW}(A) = \{a \mid S \Rightarrow \ldots Aa \ldots,\ a \in V_T\}$$

若$S \Rightarrow \ldots A$，则令$\# \in \text{FOLLOW}(A)$。

对含有归约项目的项目集I_i：

$$I_i = \{A \to \beta\bullet,\ B \to \delta\bullet,\ \ldots\ldots\}$$

填写归约动作时，首先求解FOLLOW(*A*)，FOLLOW(*B*)，再对任何输入符号*a*：

(1) 当$a \in$FOLLOW(*A*)时，置 action (*i*, *a*) ＝{按产生式 $A \to \beta$ 归约}；

(2) 当$a \in$FOLLOW(*B*)时，置 action (*i*,*a*) ＝{按产生式 $B \to \delta$ 归约}；

(3) 当*a*不属于上述情况时，不填写归约动作。

SLR(1)冲突解决方法， SLR(1)规则。

从LR(0)项目集规范族C和GO函数构造SLR(1)分析表

- 算法5.7 构造SLR(1)分析表

输入：文法G和文法G的LR(0)项目集规范族C和GO函数

输出：文法G的SLR(1)分析表

方法：

设 $C=\{I_0, I_1, \ldots, I_n\}$，每个项目集 I_k 的下标 K作为分析器的状态。

① 若GO $(I_k, a)=I_j$ $(a\in V_T)$，则置 action$(K, a)=S_j$；

- 算法5.7　构造SLR(1)分析表 （续）

② 若GO (I_k ,A)＝I_j ($A\in V_N$)，则置

GOTO(K，A)＝j ；

③ 若$A\rightarrow \alpha\cdot \in I_k$，则对所有$a\in$FOLLOW($A$)，

置action(K , a)＝r_j。

(其中假设产生式$A\rightarrow\alpha\cdot$是文法第 j 个产生式)

④ 若$S'\rightarrow S\cdot \in I_k$，则置action($K$, ＃)＝acc;

⑤ 表中空白置出错标志。

从识别活前缀的DFA构造SLR(1)分析表

- 算法5.7’ 构造SLR(1)分析表

输入：文法G及其识别活前缀的DFA。

输出：文法G的SLR(1)分析表

方法：

分析表的状态集为DFA的状态集

设为$Q=\{0, 1, 2, \ldots n\}$

① 若$f(K, a)=j$ $(a\in V_T)$，则置

action$(K, a)=S_j$；

- **算法5.7’ 构造SLR(1)分析表 （续）**

② 若$f(k, A)=j$ ($A\in V_N$)，则置

GOTO(K，A)$=j$；

③ 若$A\rightarrow \alpha\cdot \in k$，则对所有$a\in$FOLLOW($A$)，

置action(K, a)$=r_j$。

(其中假设产生式$A\rightarrow\alpha\cdot$是文法第 j 个产生式)

④ 若$S'\rightarrow S\cdot \in k$，则置action($K$, #)=acc;

⑤ 表中空白置出错标志。

- **定义5.12**

按照SLR(1)方法构造的文法*G*的LR分析表，如果每个入口不含多重定义，则称它为*G*的SLR(1)分析表。具有SLR(1)分析表的文法*G*称为SLR(1)文法。使用SLR(1)分析表的语法分析器称作SLR(1)分析器。

SLR(1)分析表构造= SLR(1)方法+ LR(0)分析表构造

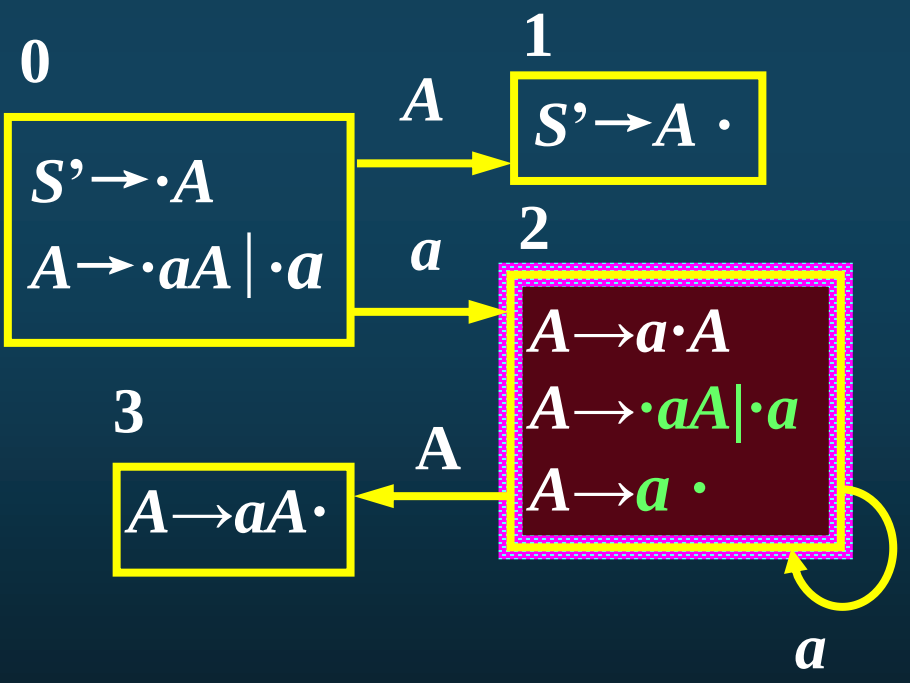
SLR(1)方法 =有归约项目的状态中归约动作的设置。

例5.14 设有文法G以及识别活前缀的DFA如下

G': $S' \to A$

$A \to aA$ ①　　FOLLOW(A) = {#}

$A \to a$ ②



G' 的SLR(1)分析表

state	action		goto
	a	#	A
0	S_2		1
1		acc	
2	S_2	r_2	3
3		r_1	

该文法为SLR(1)文法。

5.1 “移近—归约”分析法
5.2 算符优先分析法
5.3 LR分析鸟瞰
5.4 LR (0)分析
5.5 SLR (1)分析
5.6 LR (1)分析
5.7 LALR (1)分析
5.8 LR分析对二义文法的应用
5.9 LR分析的错误处理与恢复
5.10 语法分析器的自动生成与YACC

5.6.1 SLR(1)分析的缺陷

5.6.2 LR(1)分析的实现思想

5.6.3 LR (1)项目集规范族的构造

5.6.4 LR (1)分析表的构造

例5.15 设有文法$G(S)$为

(1) $S \rightarrow L=R$

(2) $S \rightarrow R$

(3) $L \rightarrow *R$

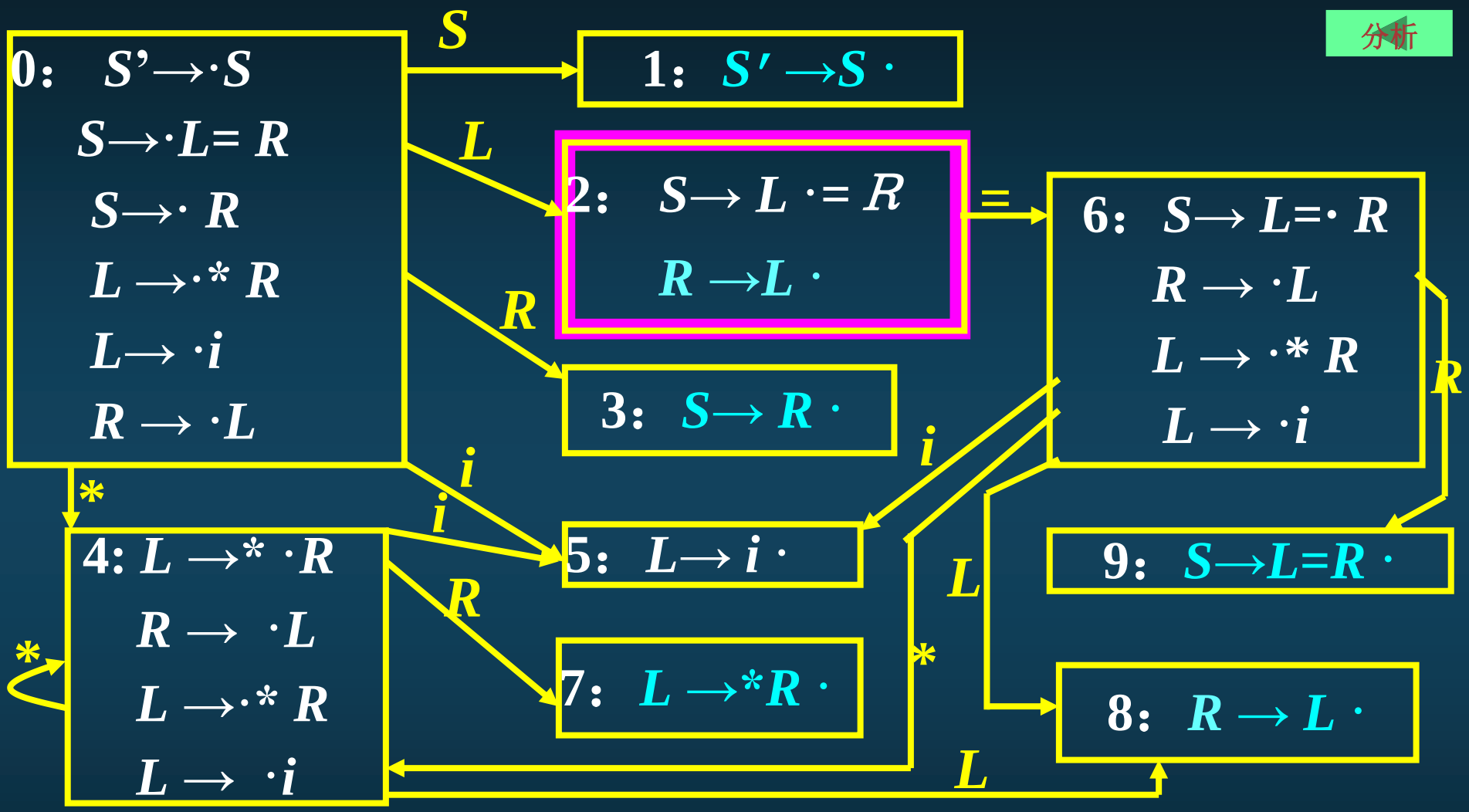
(4) $L \rightarrow i$

(5) $R \rightarrow L$

构造文法$G(S')$的识别活前缀的DFA：

开始符号的候选式不唯一，对文法进行拓广为$G(S')$，增加一产生式$S' \rightarrow S$

分析



S'→S	(1) S→ L=R	(2) S→ R
(3) L →*R	(4) L → i	(5) R → L

状态2＝ { $S \to L \cdot = R$， $R \to L \cdot$ }

存在移进－归约冲突，据SLR(1)方法求：

FOLLOW(R)= {=， # }

有　FOLLOW(R)∩ {=} $\neq \Phi$

∴ 文法G是非SLR(1)文法

对SLR(1)分析，存在 I_k ： $A \to \beta \bullet$

若 $a \in$ FOLLOW(A) ⇨ action(k , a) = {$A \to \beta$}

$S' \to S$	(1) $S \to L=R$	(2) $S \to R$
(3) $L \to *R$	(4) $L \to i$	(5) $R \to L$

$\text{action}(k, a) = \{A \to \beta\}$

归约未必有效！

∵ 可能不存在一个规范句型含有前缀δAa

∴ 用$A \to \beta$归约不一定有效。

$\text{FOLLOW}(A) = \{a \mid S \overset{*}{\Rightarrow} \ldots Aa \ldots,\ a \in V_T\}$

此推导不考虑A前面的符号串

β	$\ldots k$
δ	$\ldots$
#	0

$A \leftarrow \beta$

$: \ldots a \ldots \#$

得到$\delta Aa\ldots$形式的规范句型。

5.6.1　SLR(1)分析的缺陷

5.6.2　LR(1)分析的实现思想

5.6.3　LR (1)项目集规范族的构造

5.6.4　LR (1)分析表的构造

- **定义5.13 （ LR(k)项目）**

文法G的一个LR(k)项目是

$$[A\rightarrow\alpha\cdot\beta,\ a_1a_2\ \ldots\ a_k\ ]$$

其中：

$A\rightarrow\alpha\cdot\beta$是一个LR(0)项目；

$a_i\in V_T\cup\{\#\}$；

$a_1\ a_2\ \ldots\ a_k$：搜索符串；

- **定义5.14　（LR(1)有效项目）**

若文法G的一个LR(1)项目［$A\rightarrow \alpha\cdot\beta$，$a$］对活前缀$\gamma$是有效的，当且仅当存在规范推导

$$S \overset{*}{\underset{R}{\Rightarrow}} \delta A\omega \underset{R}{\Rightarrow} \delta\alpha\beta\omega$$

其中：$\omega\in V_T^*$，$\gamma=\delta\alpha$，$a\in$FIRST(ω)或a为’#’(当$\omega=\varepsilon$)，称a为搜索符。

例5.16：　设有文法G

$$S \rightarrow BB \qquad B \rightarrow aB \mid b$$

LR(1)项目 [$B \rightarrow a.B$, a] 对活前缀aaa有效

∵ ∃ 规范推导 $S \Rightarrow BB \Rightarrow BaB \Rightarrow Bab \Rightarrow aBab$

$$\Rightarrow \underbrace{aa}_{\delta}\underbrace{Ba}_{\omega}b \qquad \Rightarrow \underbrace{a\,a}_{\delta\alpha}\,\underbrace{a\,B}_{\alpha\beta}\,\underbrace{ab}_{\omega}$$

当$\omega=\varepsilon$

LR(1)项目 [$B \rightarrow a.B$, #] 对活前缀Baa有效

∵ ∃ 规范推导 $S \Rightarrow BB \Rightarrow BaB \Rightarrow BaaB$

DFA

例如，对例5.15 有

$I_2 = \{ S \to L \cdot = R,\ R \to L \cdot \}$

FOLLOW(R)= {=， # }

从 [$R \to L \cdot$, #] 项目考察知，它对L有效。

而[$R \to L \cdot$, =] 项目考察知，它对L无效。

$\exists S' \Rightarrow S \Rightarrow R \Rightarrow L$

$\exists S' \Rightarrow S \Rightarrow L = R$

等号前不可能只出现一个R

即不可能有规范句型$R = \omega$

$S' \to S$	(1) $S \to L=R$	(2) $S \to R$
(3) $L \to *R$	(4) $L \to i$	(5) $R \to L$

- **LR(0)有效项目**

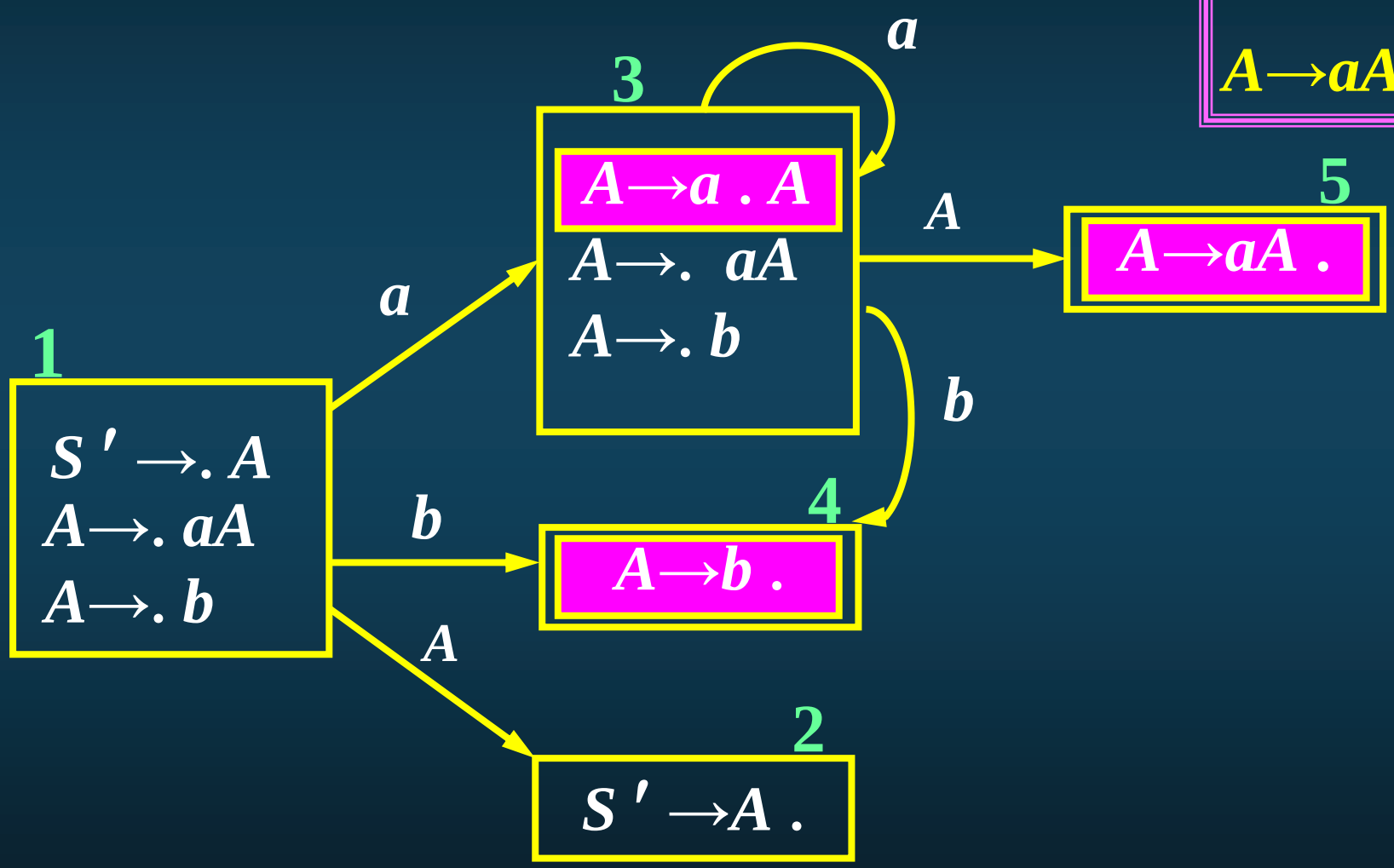
若文法G的一个LR(0)项目［$A\rightarrow \beta_1 \cdot \beta_2$］对活前缀$\delta\beta_1$是有效的，当且仅当存在规范推导

$$S \overset{*}{\underset{R}{\Rightarrow}} \delta A\omega \overset{*}{\underset{R}{\Rightarrow}} \delta\beta_1\beta_2\omega$$

例如，对例5.12的文法G(S ′)有识别活前缀的DFA如下：

$S' \to A$

$A \to aA \mid b$



5.6.1 SLR(1)分析的缺陷

5.6.2 LR(1)分析的实现思想

5.6.3 LR (1)项目集规范族的构造

5.6.4 LR (1)分析表的构造

- 构造LR(1)项目集规范族C

需要的两个函数

1：函数 closure(*I*) ;

2：GO函数 ;

- 算法5.7

*** 注意：待约项目扩展的项目的搜索符的求法**

closure (I)

{

do {

if (对 I 的每个项目 [$A \to \alpha \cdot B\beta, a$], 及G'中的每个产生式 $B \to \gamma$和 FIRST(βa)的每个符号 b , 如果[$B \to \bullet\gamma$， b]不在I中)

则把 [$B \to \bullet\gamma, b$] 加到I中 ;

} while (没有更多的项目可以加入 I) ;

return I ;

}

- **算法5.8**

GO (*I*, *X*)

{

令$J=\{[A\rightarrow \alpha X\bullet\beta, a] \mid [A\rightarrow \alpha\bullet X\beta, a]\in I\}$;

return closure(*J*);

}

LR(1)项目集*I*的GO函数：GO(*I*, *X*)是*I*中LR(0)的项目圆点右移一个位置的项目且搜索符不变，加入的项目若是待约项目则对该项目求closure。

- 算法5.9

items(G') //LR(1)的C的构造

{

C= closure ({$S' \rightarrow \bullet S$, # }); //初始化

do {

if (对C的每个项目I和每个文法符号

X, 若GO(I, X) 非空且不在C中)

把GO(I, X) 加入C中 ;

} while (没有更多的项目集可以加入C中);

}

- 算法5.9’

构造识别LR(1)项目有效活前缀的DFA

{

初态0= closure ({$S' \rightarrow \bullet S$, # }); //初始化

do {

if (对每个状态i和每个文法符号

X, 若GO(i, X)＝j 非空且不在DFA中)

j加入DFA中且$f(i, X)=j$;

} while (没有更多的状态可以加入DFA中);

}

例5.15 $S' \rightarrow S$

(1) $S \rightarrow L=R$

(2) $S \rightarrow R$

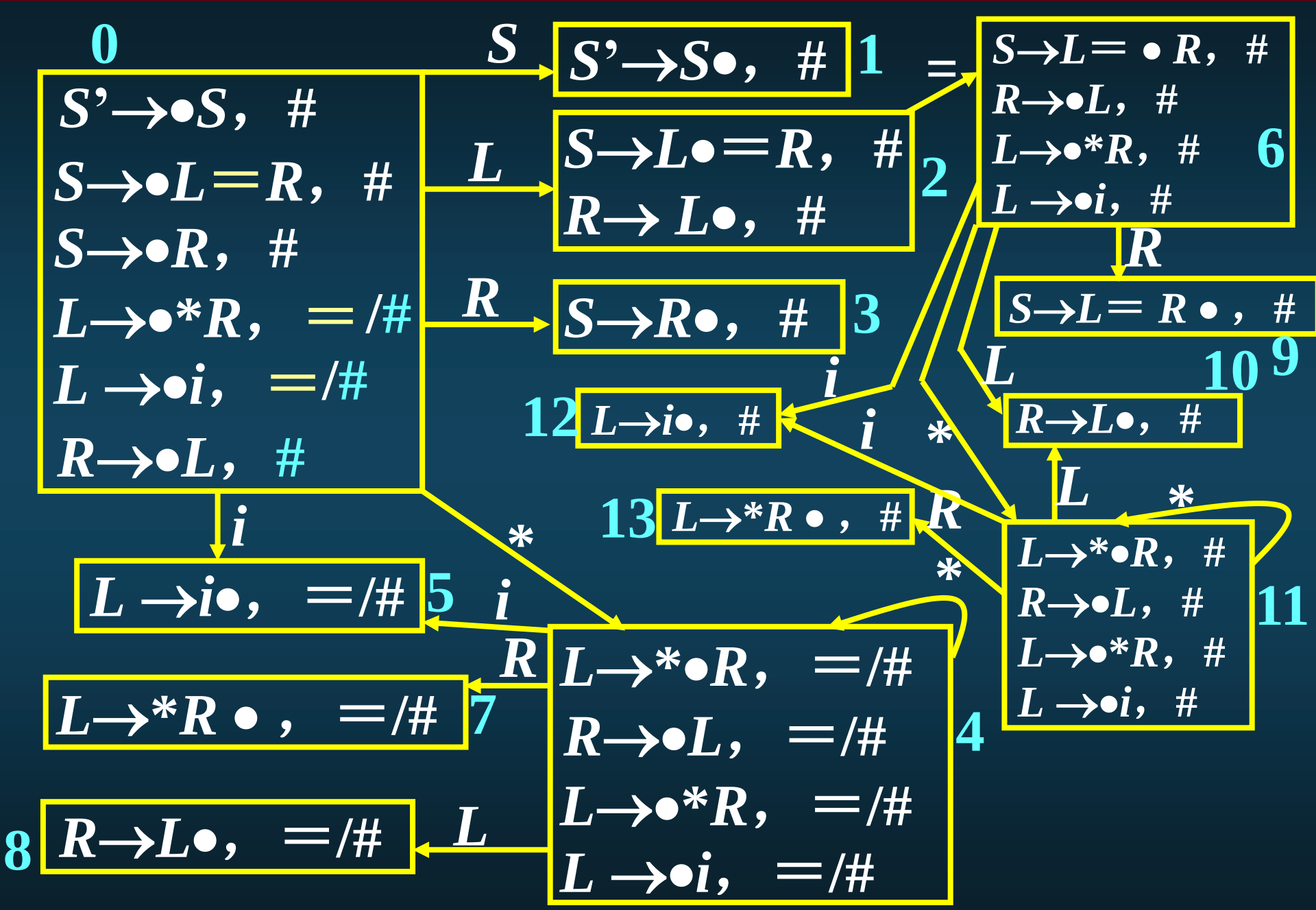
(3) $L \rightarrow *R$

(4) $L \rightarrow i$

(5) $R \rightarrow L$

构造识别LR(1)项目有效活前缀的DFA。

S'→S ①S→ L=R ②S→ R ③L →*R ④L → i ⑤ R → L



5.6.1 SLR(1)分析的缺陷

5.6.2 LR(1)分析的实现思想

5.6.3 LR (1)项目集规范族的构造

5.6.4 LR (1)分析表的构造

- **算法5.10 (LR(1)分析表构造)**

输入： 拓广的文法G'和文法G'的LR(1)项目集规范族C 和GO函数

输出： 文法G'的LR(1)分析表

方法： 设G'的LR(1)项目集规范族

$$C=\{I_0, I_1,\ldots, I_n\}$$

令每个I_k的下标 K为分析表的状态。 并含有［$S' \rightarrow \cdot S$，＃］的项目集为分析表的初态。

则有：

算法5.10 （续）

① 对于每个项目集I_i中形如［$A\rightarrow\alpha\cdot X\beta$，$b$］的项目，若GO ($I_i$, X)＝I_j，且$X\in V_T$，置action［i , X ］＝S_j。

若 $X\in V_N$时，则置：GOTO［i, X］＝j。

② 若归约项目［$A\rightarrow\alpha\cdot$, a］$\in I_i$，$A\rightarrow\alpha$为文法的第j个产生式，则置 action ［i, a］=r_j。

③ 若接受项目［$S'\rightarrow S\cdot$，#］$\in I_i$，则置：

action［i，#］= “acc”。

④ 对分析表中不能按上述规则填入信息的元素，则置“出错”标志。

- **算法5.10′ (LR(1)分析表构造)**

输入： 拓广的文法*G ′*和文法*G ′*的识别LR(1)项目有效活前缀的DFA。

输出： 文法*G ′*的LR(1)分析表

方法： 设*G ′*的识别LR(1)项目有效活前缀的DFA的

$$Q = \{0, 1, \ldots, n\}$$

令*Q*的每个状态 *K*为分析表的状态。 令初始状态0为分析表的初态。

则有：

算法5.10’（续）

① 对于每个状态K，若$f(K, a)=j$ ($a\in V_T$)，

则置action(K, a)＝S_j；若$f(k, A)=j$ ($A\in V_N$)，

则置GOTO(K，A)＝j；

②若归约项目［$A\rightarrow\alpha\cdot, a$］$\in k$，则置action[$K$, a]=r_j。

(其中假设产生式$A\rightarrow\alpha\cdot$是文法第j个产生式)

③ 若接受项目［$S'\rightarrow S\cdot$，#］$\in k$，则置

action(K, #)＝acc;

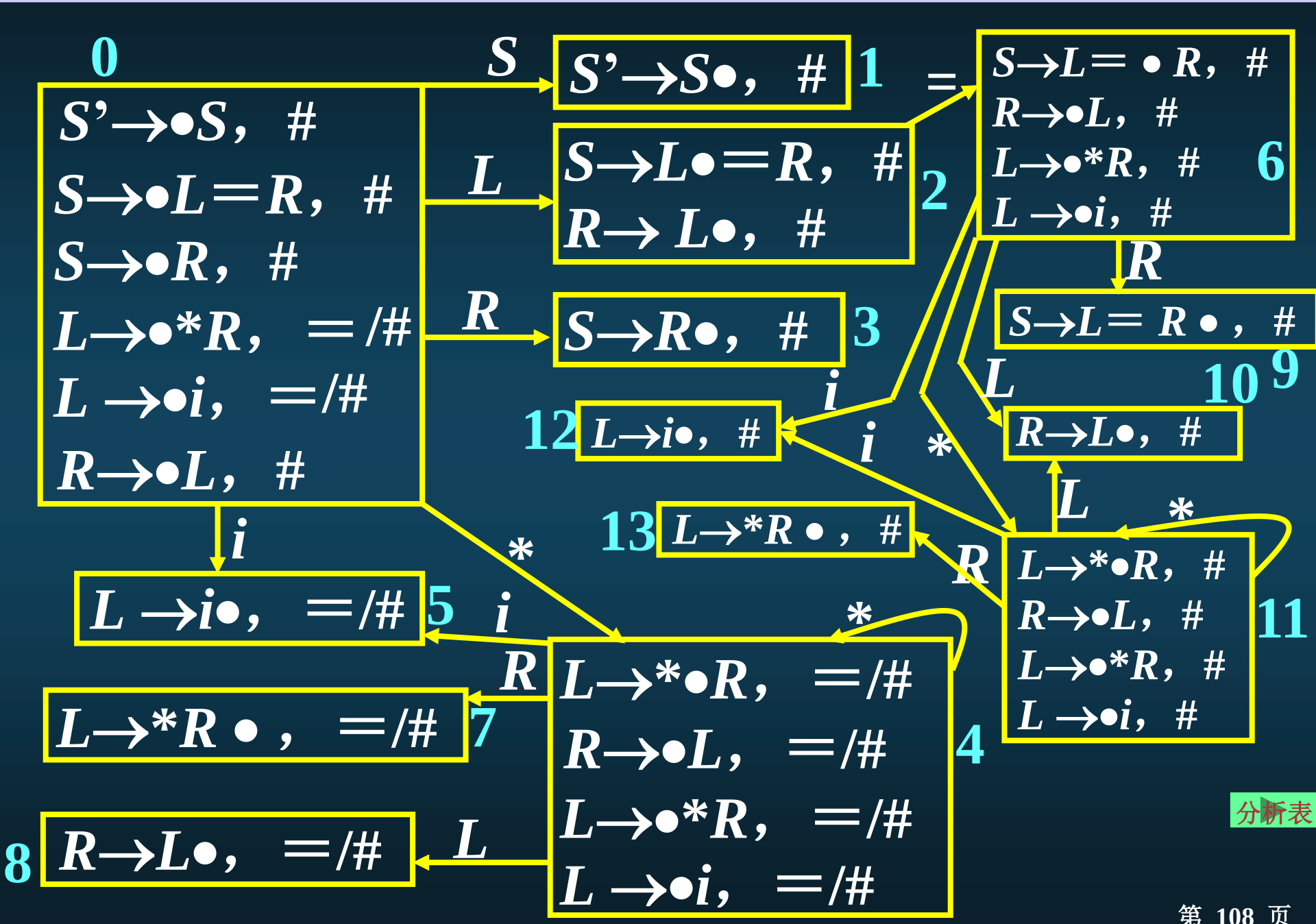
④ 对分析表中不能按上述规则填入信息的元素，

则置“出错”标志。

- 定义5.15

按照LR(1)的项目集规范族构造的文法*G*的LR (1)分析表，如果每个入口不含多重定义，则称它为*G*的LR(1)分析表。具有LR(1)分析表的文法*G*称为LR(1)文法。使用LR(1)分析表的语法分析器称作LR(1)分析器。

S'→S ①S→ L=R ②S→ R ③L →*R ④L → i ⑤ R → L



LR(1)分析表

DFA

state	action				goto		
	=	*	i	#	L	R	S
0		S_4	S_5		2	3	1
1				acc			
2	S_6			r5			
3				r2			
4		S_4	S_5		8	7	
5	r4			r4			
6		S_{11}	S_{12}		10	9	
7	r3			r3			
8	r5			r5			
9				r1			
10				r5			
11		S_{11}	S_{12}		10	13	
12				r4			
13				r3			

例5.17 设有文法$G(A)$

(1) $A' \rightarrow A$

(2) $A \rightarrow BB$

(3) $B \rightarrow aB$

(4) $B \rightarrow b$

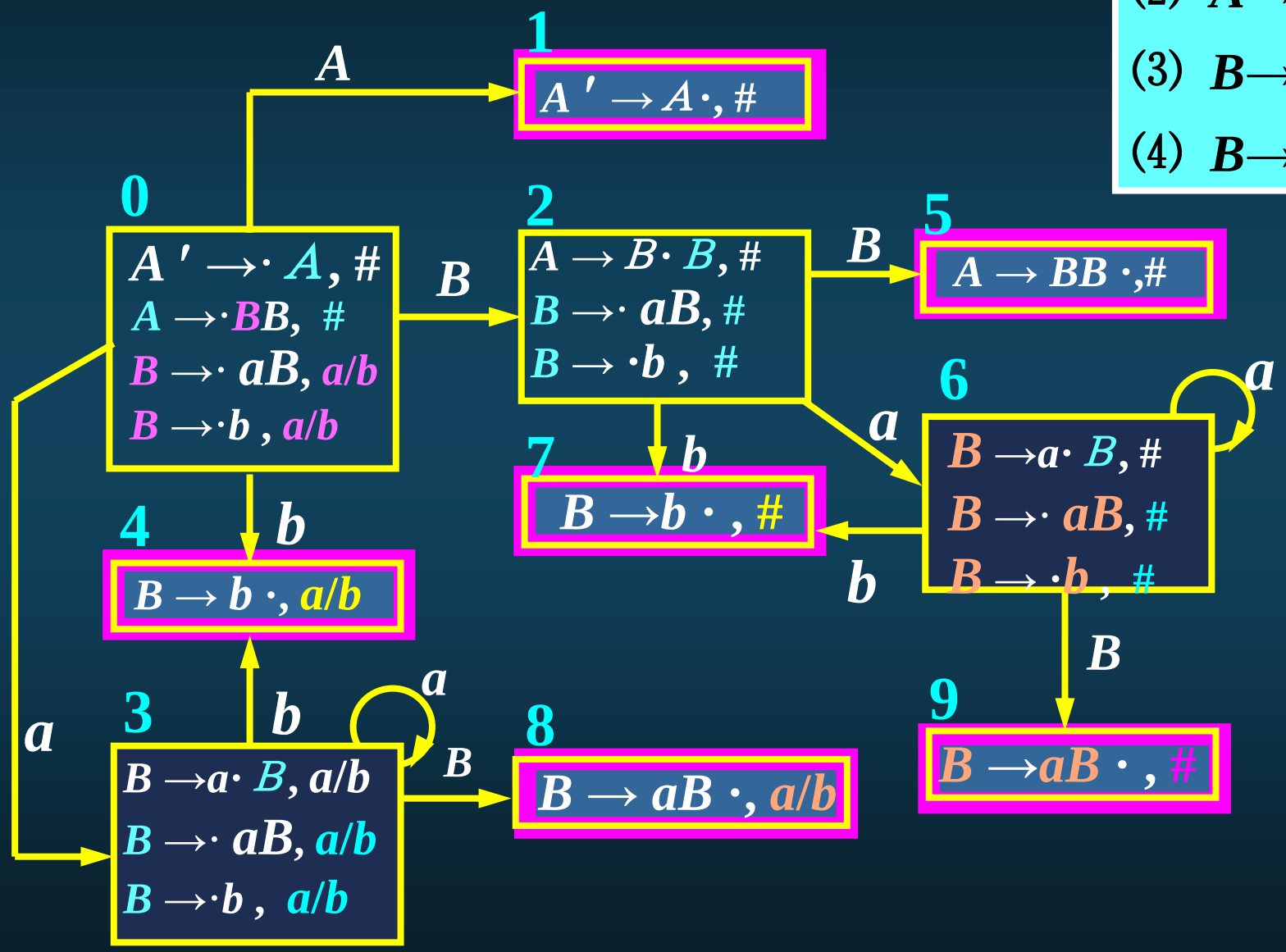
文法G(A)的识别关于LR(1)项目活前缀的DFA

(1) $A' \to A$

(2) $A \to BB$

(3) $B \to aB$

(4) $B \to b$



分析表

LR(1)分析表

state	ACTION表			GOTO表	
	a	*b*	#	*A*	*B*
0	S_3	S_4		1	2
1			acc		
2	S_6	S_7			5
3	S_3	S_4			8
4	r_4	r_4			
5			r_2		
6	S_6	S_7			9
7			r_4		
8	r_3	r_3			
9			r_3		

文法G(A)的LR(1)项目集规范族

(1) $A' \to A$

(2) $A \to BB$

(3) $B \to aB$

(4) $B \to b$

$\{A' \to \cdot A, \#; A \to \cdot BB, \#; B \to \cdot aB, a/b; B \to \cdot b, a/b\}$ I0

$\{A' \to A \cdot, \#\}$ I1

$\{A \to B \cdot B, \#; B \to \cdot aB, \#; B \to \cdot b, \#\}$ I2

$\{B \to a \cdot B, a/b; B \to \cdot aB, a/b; B \to \cdot b, a/b\}$ I3

$\{B \to b \cdot, a/b\}$ I4

$\{A \to BB \cdot, \#\}$ I5

$\{B \to a \cdot B, \#; B \to \cdot aB, \#; B \to \cdot b, \#\}$ I6

$\{B \to b \cdot, \#\}$ I7

$\{B \to aB \cdot, a/b\}$ I8

$\{B \to aB \cdot, \#\}$ I9

分析表

GO函数

	a	b	A	B
0	3	4	1	2
2	6	7		5
3	3	4		8
6	6	7		9

1,4,5,7,8,9是归约态。

LR(1)分析表

state	ACTION表			GOTO表	
	a	*b*	#	*A*	*B*
0	S_3	S_4		1	2
1			acc		
2	S_6	S_7			5
3	S_3	S_4			8
4	r_4	r_4			
5			r_2		
6	S_6	S_7			9
7			r_4		
8	r_3	r_3			
9			r_3		

end